

Semaglutide PGX Digital Form System

Product Requirements Document (PRD)

Version 1.0 | May 2026

CONFIDENTIAL

Prepared for: Vendor / Development Team

Prepared by:

1. Executive Summary

OneDNA is developing a secure, full-stack web application to digitise the Semaglutide Pharmacogenomics (PGX) Test Requisition Form. The platform replaces paper-based workflows with a structured, multi-step digital form experience accessible to hospital-side clients (Users) and centrally monitored by an Administrator.

The system will enable hospital-side Users to complete the requisition form section by section, save progress as drafts across sessions, preview all entered data before final submission, and submit with a patient declaration. Administrators receive real-time WebSocket notifications upon submission, can view all submitted forms categorised by client, and manage the user base through a dedicated control panel.

This document constitutes the complete Product Requirements Document (PRD) intended for use as a formal vendor engagement and development proposal.

2. Project Scope & Objectives

2.1 In-Scope

- Multi-step, section-wise digital form mirroring the Semaglutide PGX Test Requisition Form
- Draft save and resume functionality across multiple sessions
- Patient declaration and final submission with confirmation
- Admin dashboard with analytics (daily, weekly, monthly, yearly submissions)
- Admin user management — create, list users with RBAC enforcement
- Data table view per client/user with detailed form information modal
- Real-time WebSocket notification to Admin on form submission
- Role-based access control (RBAC): Admin role and User role
- Responsive frontend (mobile, tablet, desktop)
- Secure REST API backend with enterprise-level architecture

2.2 Out-of-Scope

- Mobile native application (iOS / Android)
- Third-party EHR / LIS integrations
- Automated report generation or PDF report output
- Payment or billing workflows
- Multi-language / localisation support

3. Stakeholders

Stakeholder	Role	Responsibilities
OneDNA Product Team	Product Owner	Define requirements, approve deliverables
Hospital Client / User	End User	Complete and submit the Semaglutide PGX form

Stakeholder	Role	Responsibilities
OneDNA Admin	Administrator	Manage users, monitor submissions, review data
Development Vendor	Implementation Partner	Design, develop, deploy, and document the system

4. System Architecture Overview

4.1 High-Level Architecture

The system follows a two-tier client-server architecture with a single-page frontend and a RESTful API backend. Real-time capabilities are handled via native WebSocket protocol. Persistent data is stored in a PostgreSQL database hosted on Supabase.

Layer	Technology	Deployment
Frontend	Next.js 14+ (App Router), TypeScript, Tailwind CSS, shadcn/ui, MUI, Zustand	Vercel
Backend API	Fastify, TypeScript, Zod (validation)	Digital Ocean (Droplet / App Platform)
Real-Time Layer	Native WebSocket (ws library, no Socket.io)	Co-hosted with API server
Database	PostgreSQL via Supabase	Supabase Cloud
Authentication	JWT (access + refresh tokens), bcrypt for password hashing	Backend-managed

4.2 Backend Architecture

The Fastify backend shall follow a strict layered architecture:

- Routes — HTTP method + path definition, minimal logic
- Controllers — Request parsing, response shaping, delegation to services
- Services — Business logic, orchestration of multiple repositories
- Repositories — All direct database queries, parameterised and typed
- Middleware — Authentication guard, RBAC enforcement, request logging, error handling
- Validators — Zod schemas co-located with each feature module
- Config — Environment variables via dotenv, validated at startup

4.3 RBAC Middleware

Every protected route shall pass through two middleware layers:

1. `verifyToken` — Validates the JWT, attaches decoded user payload to request context
2. `requireRole(role[])` — Checks the decoded role against the allowed roles for that route; returns HTTP 403 if unauthorised

Admin credentials are hardcoded in a secure environment variable file and seeded into the database at first boot. No self-registration is permitted; all User accounts are created exclusively by the Admin.

5. User Roles & Permissions

Role	Created By	Authentication	Accessible Areas
Admin	System seed (hardcoded)	UserID + Password	Dashboard, Data, User Create, Notifications
User (Client)	Admin only	UserID + Password	New Entry Form, My Drafts

On the login page, the user selects their role (Admin or User) before entering credentials. The system matches credentials against the appropriate table and redirects to the corresponding dashboard. Role selection at login provides a UX hint and allows the server to short-circuit credential lookup.

6. Detailed Feature Requirements (note: refer to the sample pdf for more information)

6.1 Authentication & Login

- Single login page with role selector (Admin / User)
- Fields: UserID, Password
- On success: JWT access token (15 min expiry) and refresh token (7 days) issued
- Tokens stored in HttpOnly cookies — no localStorage usage
- Automatic token refresh via silent refresh on expiry
- Logout clears cookies and invalidates the refresh token server-side
- Failed login returns a generic error message (no user enumeration)

6.2 User-Side: Multi-Step Form (New Entry)

The form mirrors the physical Semaglutide PGX Test Requisition Form and is divided into the following sections presented as discrete steps, with a side navigation panel indicating progress:

#	Section Name	Key Fields	Field Types
1	Personal Details	Full Name, DOB, Age, Gender, Phone, Email, State/City, Ethnicity, Primary Indication	Text, Date, Dropdown, Radio, Email, Phone
2	Anthropometrics & Vitals	Height, Weight, BMI (auto-calculated), Waist Circumference, Blood Pressure, Heart Rate, SpO2	Number, Auto-calc, Text
3	Metabolic & Biochemistry Results	HbA1c, FPG, Postprandial Glucose, Fasting Insulin,	Number with unit labels, Optional fields

#	Section Name	Key Fields	Field Types
		HOMA-IR, C-Peptide, Lipid Panel, Liver Enzymes, Creatinine, eGFR, TSH, Vitamins, CRP, Albumin, Uric Acid, CBC	
4	Diabetes & Metabolic History	Duration of T2DM, Year of Diagnosis, Family History, Prior Anti-Diabetic Agents, Prior GLP-1 Agonist Details	Radio, Checkbox, Text, Nested conditional fields
5	Comorbidities & Medical History	Hypertension, Dyslipidaemia, Retinopathy, CAD, Stroke, PCOS, Prior Pancreatitis, GERD, Thyroid Cancer, Other; Hospitalisation notes; USG findings; Fundus	Checkbox grid, Text area, Radio
6	Current Medications	Drug Name, Dose, Frequency, Indication; Supplements/OTC; Known Drug Allergies	Repeatable row table, Text, Radio (None)
7	Lifestyle & Environmental Factors	Occupation, Daily Steps, Exercise Type/Duration, Sleep Duration/Quality, Smoking, Alcohol, Stress Level, Dietary Pattern, Occupational Activity	Text, Number, Radio, Checkbox, Dropdown
8	Pharmacogenomics & Sample Information	Prior Genetic Testing, Sample Type, Kit ID, Date & Time of Collection, Referring Physician	Radio, Checkbox, Text, DateTime
9	Clinician Preliminary Assessment	ASCVD Risk, GI Tolerability Risk, Pancreatitis Risk, Absolute Contraindications, Proposed Starting Dose, Target Maintenance Dose, Additional Observations	Radio (Low/High), Checkbox, Text area, Dropdown
10	Key Variants — Genotyping	Gene/Variant, Genotype/Status, Method, Notes for GLP1R, TCF7L2, KCNQ1, ADRB3, FTO genes	Text, Dropdown, Editable table rows
11	Preview & Declaration	Read-only summary of all sections; Patient consent statement; Patient Signature, Date & Time	Read-only display, Checkbox (consent), Date, Signature input

6.3 Draft Save & Resume

- A 'Save as Draft' button is available on every section step
- Draft data is persisted to the backend database, linked to the logged-in user
- The Drafts page lists all saved drafts with: Form Reference ID, Patient Name, Date Created, Last Updated, Completion Status (percentage), and a 'Continue' action
- Clicking 'Continue' opens the form at the exact section where the user last saved
- Multiple concurrent drafts per user are permitted

- Upon final submission, the corresponding draft record is automatically deleted
- Auto-save (every 2 minutes of inactivity) is recommended to prevent data loss

6.4 Form Submission

- Submission is only enabled after the user checks the Patient Consent & Declaration checkbox and provides a signature on the final preview step
- On Submit: form data is validated server-side, persisted to the submissions table, associated draft is deleted, and a WebSocket event is emitted to all connected Admin sessions
- User receives a success confirmation screen with a Submission Reference ID

6.5 Admin: Real-Time Notifications (WebSocket)

- The backend maintains a WebSocket server using the native 'ws' Node.js library — Socket.io is explicitly excluded
- On Admin login, the frontend establishes a persistent WebSocket connection authenticated via JWT
- On each form submission, the server pushes a notification payload to all authenticated Admin WebSocket connections
- Notification payload: { type: 'FORM_SUBMITTED', submissionId, patientName, submittedBy, timestamp }
- The Admin UI displays a notification bell counter and a notification panel listing recent submissions
- Clicking a notification navigates directly to the detailed submission view

6.6 Admin: Dashboard

- Summary metric cards: Total Submissions (All Time), Submissions This Day / Week / Month / Year — with a period filter toggle
- Recent Submissions table: Submission ID, Patient Name, Submitted By (User), Date, Action (View Details)
- Charts: Bar chart of submissions over time (period controlled by filter)
- All counts update in real time when a new WebSocket submission event is received

6.7 Admin: Data View

- Sidebar 'Data' navigates to a page listing all Users/Clients who have submitted at least one form
- For each User/Client, a distinct table section is rendered displaying all forms submitted by that user
- Each table row shows: Form Reference ID, Patient Name, Collection Date, Referring Physician, Status, and a 'More Info' button
- 'More Info' opens a full-page detail view (or modal) rendering all eleven form sections in a read-only, well-labelled layout

6.8 Admin: User Management (User Create)

The Admin can create new hospital-side User accounts via the 'User Create' sidebar section:

- View all registered users in a table with: Name, UserID, Phone, Address, Date Created, Status
- 'Add User' button opens a form with fields: Full Name, UserID, Password, Confirm Password, Contact Number, Address
- Server validates uniqueness of UserID before creation

- Created users can immediately log in with the provided credentials
- Admin can deactivate (soft-delete) users — no hard deletes permitted

7. Non-Functional Requirements

Category	Requirement
Performance	API response time < 300 ms (p95) under normal load. Frontend initial load (LCP) < 2.5 s on 4G.
Security	HTTPS enforced end-to-end. JWT HttpOnly cookies. Input sanitisation + Zod validation on all endpoints. Rate limiting on login (max 5 attempts / 15 min per IP). SQL injection prevention via parameterised queries. CORS restricted to whitelisted origins.
Scalability	Stateless API design. Database connection pooling. WebSocket connections must not block HTTP workers.
Availability	99.5% uptime SLA. Zero-downtime deployments via rolling updates.
Data Integrity	All draft and submission data is persisted transactionally. No partial writes permitted.
Auditability	All API requests logged with timestamp, user ID, route, and HTTP status. Logs retained for 90 days.
Responsiveness	UI is fully responsive across mobile (360 px+), tablet (768 px+), and desktop (1280 px+). Uses CSS Grid and Flexbox throughout.
Accessibility	WCAG 2.1 Level AA compliance. Semantic HTML, ARIA labels, keyboard navigation support.

8. Database Schema (note: refer to the sample pdf for more information)

8.1 Core Tables

Table	Column	Type	Description
users	id	UUID (PK)	Unique user identifier
	user_id	VARCHAR(50) UNIQUE	Login username
	name	VARCHAR(150)	Full name
	password_hash	TEXT	bcrypt hashed password
	role	ENUM('admin','user')	Access role
	phone	VARCHAR(20)	Contact number
	address	TEXT	Physical address
	is_active	BOOLEAN	Soft-delete flag
	created_at	TIMESTAMPTZ	Account creation time

Table	Column	Type	Description
form_drafts	id	UUID (PK)	Draft identifier
	user_id	UUID (FK → users)	Owning user
	current_section	SMALLINT	Last saved step (1–11)
	form_data	JSONB	All section data as JSON
	updated_at	TIMESTAMPTZ	Last auto-save or save timestamp
form_submissions	id	UUID (PK)	Submission identifier
	user_id	UUID (FK → users)	Submitting user
	reference_no	VARCHAR(30) UNIQUE	Human-readable reference
	form_data	JSONB	Complete submitted form data
	submitted_at	TIMESTAMPTZ	Submission timestamp
notifications	id	UUID (PK)	Notification record
	submission_id	UUID (FK)	Related submission
	is_read	BOOLEAN	Admin read status
	created_at	TIMESTAMPTZ	Notification creation time

9. API Endpoint Specification

Method	Path	Auth Required	Description
POST	/api/auth/login	None	Login — returns JWT access + refresh tokens
POST	/api/auth/refresh	Refresh token (cookie)	Issue new access token
POST	/api/auth/logout	Access token	Invalidate refresh token
GET	/api/admin/users	Admin	List all users
POST	/api/admin/users	Admin	Create new user
PATCH	/api/admin/users/:id	Admin	Update user (deactivate / edit)
GET	/api/drafts	User	List all drafts for logged-in user
GET	/api/drafts/:id	User	Get specific draft by ID
POST	/api/drafts	User	Create new draft
PATCH	/api/drafts/:id	User	Update draft data + current section
DELETE	/api/drafts/:id	User	Delete a draft

Method	Path	Auth Required	Description
POST	/api/submissions	User	Submit completed form (deletes draft, triggers WS event)
GET	/api/admin/submissions	Admin	List all submissions with filters (date range, user)
GET	/api/admin/submissions/:id	Admin	Get full submission detail
GET	/api/admin/submissions/stats	Admin	Get counts grouped by day/week/month/year
GET	/api/admin/notifications	Admin	List unread + recent notifications
PATCH	/api/admin/notifications/:id/read	Admin	Mark notification as read
WS	/ws	JWT query param	Admin real-time notification channel

10. Frontend Specification

10.1 Design System

Token	Value	Usage
Primary Background	#000000 / #0A0A0A (60%)	Page background, sidebar, cards
Text Primary	#FFFFFF (30%)	All primary text, headings
Accent / Highlight	#00BCD4 Cyan (10%)	Borders, active states, CTAs, focus rings
Typography	Inter / System-UI sans-serif	All UI text
Border Radius	8 px (cards), 4 px (inputs)	All interactive elements
Spacing Unit	4 px base (Tailwind scale)	Padding, margins, gaps

10.2 Page Structure

User-Side Pages:

- / login — Role selector + credential form
- / dashboard — Redirect to /drafts or /new-entry
- / new-entry — Multi-step form (11 sections + side nav progress indicator)
- / drafts — List of saved drafts with continue and delete actions
- / submission-success — Confirmation screen with reference ID

Admin-Side Pages:

- / admin/dashboard — Metrics, recent submissions, chart
- / admin/data — Per-user submission tables with 'More Info' navigation

- / admin/data/submissions/:id — Read-only full detail view
- / admin/users — User list with 'Add User' capability
- Notification panel — Slide-over or dropdown from top nav bell icon

10.3 State Management (Zustand)

- authStore — Token, user ID, role, login/logout actions
- formStore — Active form data per section, current step, draft ID
- notificationStore — WebSocket connection instance, notification list, unread count
- uiStore — Sidebar open state, modal states, filter selections

10.4 Form Step Navigation

- Sidebar displays numbered steps with status: Not Started, In Progress, Completed
- Next Step button validates current section via client-side Zod schema before advancing
- Previous button navigates back without validation
- Save as Draft persists current state to API and shows a toast notification
- All steps completed + declaration accepted → Submit button becomes active

11. Deployment & Infrastructure

Component	Configuration
Frontend (Vercel)	Next.js App deployed to Vercel. Environment variables managed via Vercel project settings. Preview deployments for feature branches.
Backend (Digital Ocean)	Fastify app deployed as a Docker container on a Digital Ocean Droplet or App Platform. Process managed by PM2 or Docker Compose. Nginx reverse proxy with TLS termination.
Database (Supabase)	PostgreSQL hosted on Supabase. Connection via PgBouncer pooler URL. RLS (Row Level Security) policies enabled as a secondary defence. Database backups enabled (daily, 7-day retention).
Environment Variables	DATABASE_URL, JWT_ACCESS_SECRET, JWT_REFRESH_SECRET, ADMIN_SEED_ID, ADMIN_SEED_PASSWORD_HASH, CORS_ORIGINS, NODE_ENV. No secrets committed to version control.
CI/CD	GitHub Actions pipeline: lint → type-check → unit tests → build → deploy. Frontend auto-deploys from main branch. Backend deploys after manual approval gate.

12. Security Requirements

- All API communication over HTTPS (TLS 1.2 minimum)
- JWT stored exclusively in HttpOnly, Secure, SameSite=Strict cookies
- Passwords hashed with bcrypt (minimum 12 salt rounds) — never stored in plain text
- Zod validation on every API request body and query parameter
- Rate limiting on /api/auth/login endpoint (5 attempts per 15 minutes per IP)

- CORS policy restricted to whitelisted frontend origin(s) only
- All database queries parameterised to prevent SQL injection
- Sensitive environment variables never committed to source control (use .env.example for documentation)
- Admin seed credentials provided only via environment variable, not hardcoded in source files
- WebSocket connections authenticated via JWT passed as a query parameter on handshake; unauthenticated connections are immediately closed
- Request logging excludes sensitive field values (passwords, tokens)

13. Acceptance Criteria

13.1 Authentication

3. Admin and User can log in with correct credentials and are redirected to their respective dashboards
4. Invalid credentials return an error without revealing which field is wrong
5. Accessing a protected route without a valid token returns HTTP 401
6. A User attempting to access an Admin route is rejected with HTTP 403

13.2 Form

7. All 11 sections render with correct field types matching the physical form
8. BMI auto-calculates correctly from height and weight inputs
9. 'Save as Draft' persists all entered data and resumes at the correct section
10. Mandatory fields are validated before advancing to the next step
11. Submit button is disabled until declaration checkbox is checked
12. Successful submission deletes the draft and shows the reference ID

13.3 Admin

13. Admin receives a WebSocket notification within 2 seconds of a form submission
14. Clicking a notification navigates to the correct submission detail view
15. Dashboard metrics accurately reflect the correct submission counts per period
16. Data view correctly groups all submissions by User/Client in separate tables
17. Admin can create a User who can then log in successfully

14. Glossary

Term	Definition
PGX	Pharmacogenomics — the study of how genes affect a person's response to drugs
Semaglutide	A GLP-1 receptor agonist medication used for Type 2 Diabetes and weight management
RBAC	Role-Based Access Control — access rights assigned based on user roles
JWT	JSON Web Token — a compact, URL-safe token format used for authentication

Term	Definition
Draft	A partially completed form that has been saved but not yet submitted
Submission	A fully completed form that has been submitted with patient declaration
WebSocket	A full-duplex communication protocol over a single TCP connection, used here for real-time notifications
Zod	A TypeScript-first schema validation library used for API request validation
Zustand	A lightweight state management library for React applications
Supabase	An open-source Firebase alternative providing PostgreSQL hosting, auth, and storage